

Figure 14.7 Simple Use of Public-Key Encryption to Establish a Session Key

Symmetric Key Distribution using Asymmetric Encryption — Short Notes

Goal:

Securely share a **secret session key (Ks)** using **public key encryption**.

✓ Steps:

1. **A → B:** Sends public key PU_a and identity ID_a
2. **B → A:**
 - Generates session key K_s
 - Encrypts K_s with PU_a : $E(PU_a, K_s)$
 - Sends it to A
3. **A** decrypts using private key PR_a : $D(PR_a, E(PU_a, K_s)) = K_s$
4. Now both A and B share K_s for secure communication.

✓ Advantages:

- No key exists before/after communication
- Secure against **eavesdropping**

✗ Weakness – Man-in-the-Middle Attack:

- Attacker E can intercept and replace keys
- Both A and B get same K_s
- E can read all messages secretly

🔑 Tip for Diagram:

Label arrows as:

- (1) $PU_a || ID_a$
- (2) $E(PU_a, K_s)$

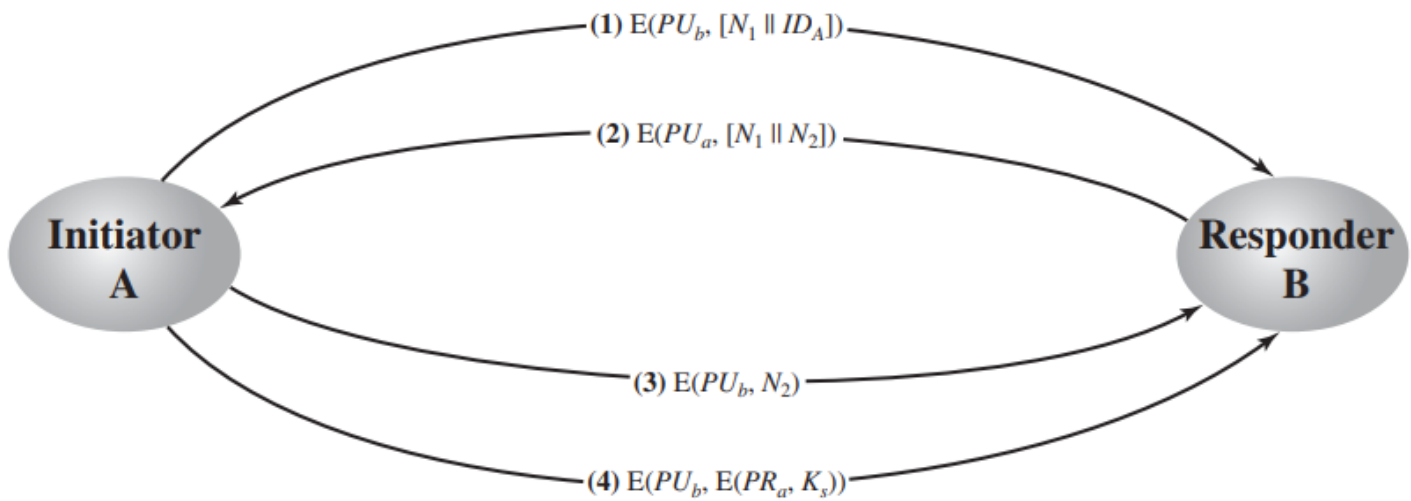


Figure 14.8 Public-Key Distribution of Secret Keys

Secret Key Distribution with Confidentiality & Authentication (Short Notes)

Goal:
Securely share session key K_s between A and B, with both **confidentiality** and **authentication**.

Steps:

1. **A → B:**
 $E(PU_b, [N1 || ID_A])$
 → A sends a random nonce $N1$ and identity, encrypted with B's public key.
2. **B → A:**
 $E(PU_a, [N1 || N2])$
 → B confirms receipt (by returning $N1$) and adds new nonce $N2$, encrypted with A's public key.
3. **A → B:**
 $E(PU_b, N2)$
 → A sends back $N2$, encrypted with B's public key to prove it's A.
4. **A → B:**
 $E(PU_b, E(PR_a, K_s))$
 → A sends session key K_s , first signed with A's private key, then encrypted with B's public key.

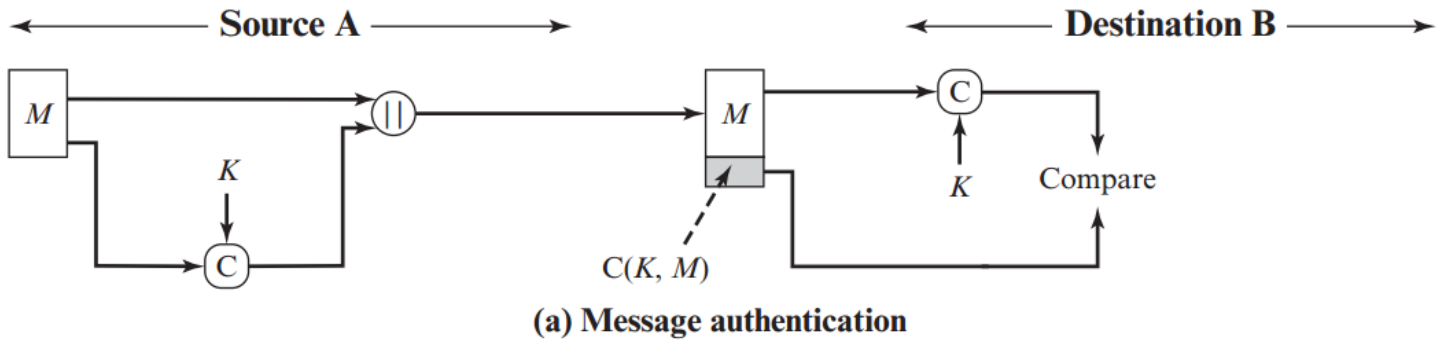
Result:

- **Confidentiality:** Only B can decrypt with PR_b
- **Authentication:** B verifies A using PU_a
- Both share K_s safely

Diagram Tip:

Draw 4 arrows with labels (1) to (4), just like shown.

Message Authentication Code (MAC)



(a) Message Authentication Only

(No encryption, only authenticity check using MAC)

💡 What it shows (Summary)

Only **authenticity** is verified — message is **not encrypted**.

Used when we only want to ensure the message wasn't **modified** or **forged**.

🧠 Steps to Memorize for Drawing

📡 Sender Side (Source A):

1. Start with **Message M**
2. Use **MAC generator C** with **Key K** → generate $C(K, M)$
3. Combine message and MAC: $[M || C(K, M)]$
4. Send both to receiver

📡 Receiver Side (Destination B):

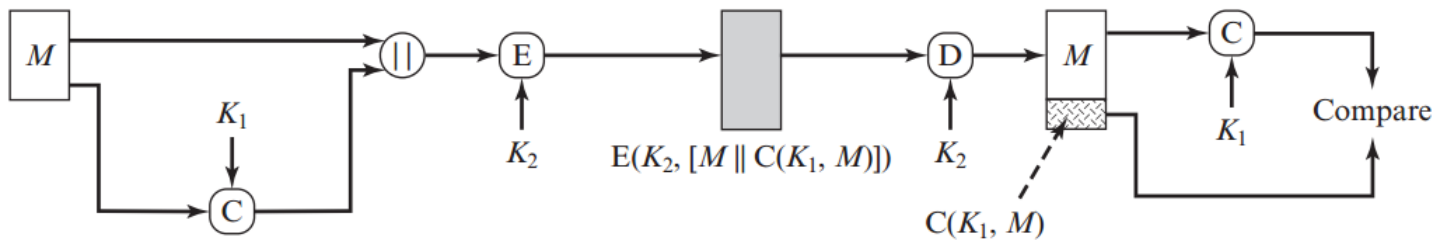
5. Receive M and $C(K, M)$
6. Use same **Key K** to recompute $C(K, M)$
7. Compare computed MAC with received MAC:
 - ☒ Match → message is authentic
 - ☐ No match → message is tampered

📐 What to Draw in Diagram:

- M goes:
 - To MAC generator C with key K → gives $C(K, M)$
 - To join with MAC using $||$
- $[M || C(K, M)]$ sent to receiver
- Receiver separates M and $C(K, M)$
- Recomputes MAC with C and K
- Compares → result goes to **Compare block**

📌 Shortcut to Remember:

MAC first → combine with message → send → recompute → compare



(b) Message authentication and confidentiality; authentication tied to plaintext

"Message authentication and confidentiality; authentication tied to plaintext"

✓ What the diagram shows

It combines:

- **Authentication** (via MAC)
- **Confidentiality** (via encryption)
using two **keys**: K_1 for MAC and K_2 for encryption.

🧠 Steps to Memorize for Drawing:

📡 Sender Side:

1. **Message = M**
2. **Use Key K_1** with message M to create **MAC = $C(K_1, M)$**
3. Combine both: $[M || C(K_1, M)]$
4. **Encrypt** this combination using **Key K_2** → gives: $E(K_2, [M || C(K_1, M)])$
5. Send the encrypted block to the receiver.

📡 Receiver Side:

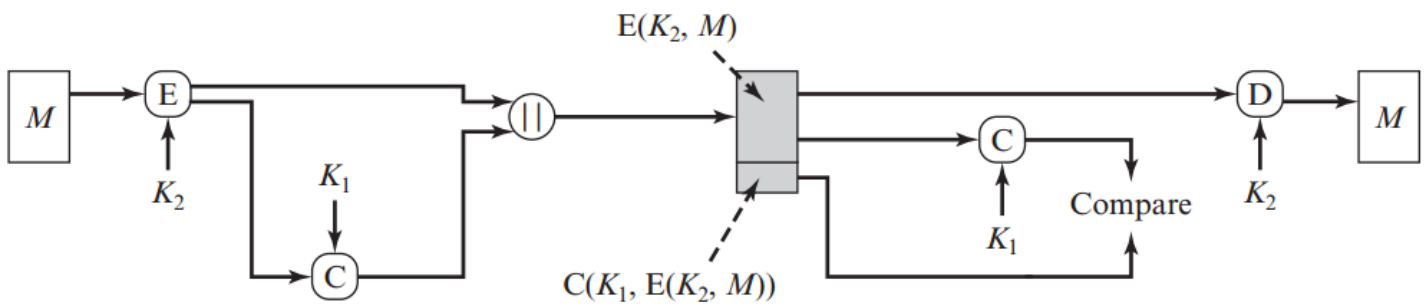
6. **Decrypt** using K_2 to get $[M || C(K_1, M)]$
7. Separate the **message M** and the **MAC**
8. Recompute MAC from received M using K_1
9. **Compare** the computed MAC with the received MAC:
 - ✓ Match → message is authentic
 - ✗ No match → message is tampered

📐 What to Draw (for Exam):

- Left: Message M goes two ways
- Down: to MAC generator C using K_1 → gives $C(K_1, M)$
- Up: Combine $M || C(K_1, M)$
- → Into Encryption block E using key K_2
- → Encrypted output goes to the Receiver
- → Decryption block D using K_2
- → Output separates to M and $C(K_1, M)$
- → Compute MAC again using C and K_1
- → Compare step

💡 Shortcut to Remember:

MAC first → combine → encrypt → send → decrypt → recompute MAC → compare



(c) Message authentication and confidentiality; authentication tied to ciphertext

"Message authentication and confidentiality; authentication tied to ciphertext"

💡 What it shows (Summary)

In this method:

- **Message M is encrypted first**
- Then **MAC is calculated over the ciphertext**
This ensures **both confidentiality and authentication** of the encrypted data.

🧠 Steps to Memorize for Drawing

📡 Sender Side:

1. Start with **message M**
2. Encrypt it using **key K_2** → gives $E(K_2, M)$
3. Use **key K_1** to generate MAC on the ciphertext:
→ $C(K_1, E(K_2, M))$
4. Combine both: $[E(K_2, M) || C(K_1, E(K_2, M))]$
5. Send both together to receiver

📡 Receiver Side:

6. Receive the encrypted message and MAC
7. Use **K_1** to recompute MAC on the ciphertext
8. Compare it with the received MAC:
 - ☒ Match → proceed to decrypt
 - ☒ No match → message is tampered
9. Use **K_2** to decrypt ciphertext and recover original M

📐 What to Draw in Diagram:

- $M \rightarrow$ into E (encryption) with $K_2 \rightarrow$ gives $E(K_2, M)$
- $E(K_2, M)$ goes:
 - One way → to be combined with MAC ($||$)
 - Other way → into MAC generator C with $K_1 \rightarrow$ gives $C(K_1, E(K_2, M))$
- Combine and send to receiver
- Receiver side:
 - Extract $E(K_2, M)$ and MAC
 - Recalculate MAC with C and K_1
 - Compare
 - If OK → decrypt using D and $K_2 \rightarrow$ get back M

📌 Shortcut to Remember:

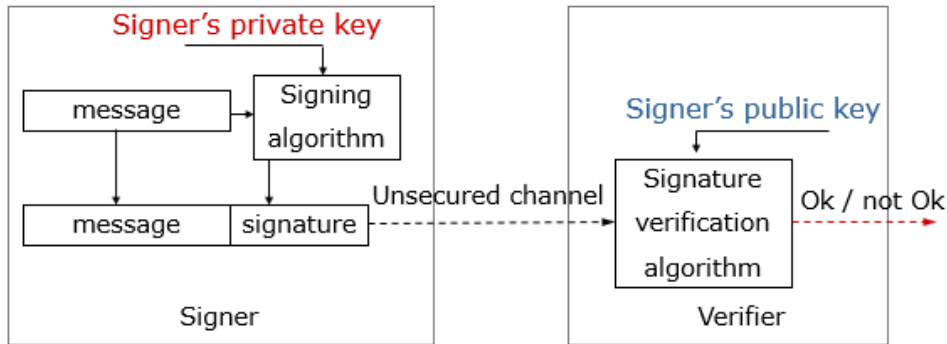
Encrypt first → then MAC the ciphertext → send → verify MAC → decrypt

Digital Signatures Scheme

❑ Used to provide

- Data integrity
- Message authentication
- Non-repudiation

Note: RSA Algorithm is normally used for producing keys to create Digital Signature



✓ Digital Signature – Short Notes

🔒 Purpose:

- Ensure **Integrity, Authentication, Non-repudiation**

👤 At Sender (Signer):

1. Message + **Private Key** → Signing Algorithm
2. Output: **Message + Signature**

👤 At Receiver (Verifier):

1. Message + Signature + **Public Key** → Verification
2. Output: **✓ OK / ✗ Not OK**

🔑 Uses:

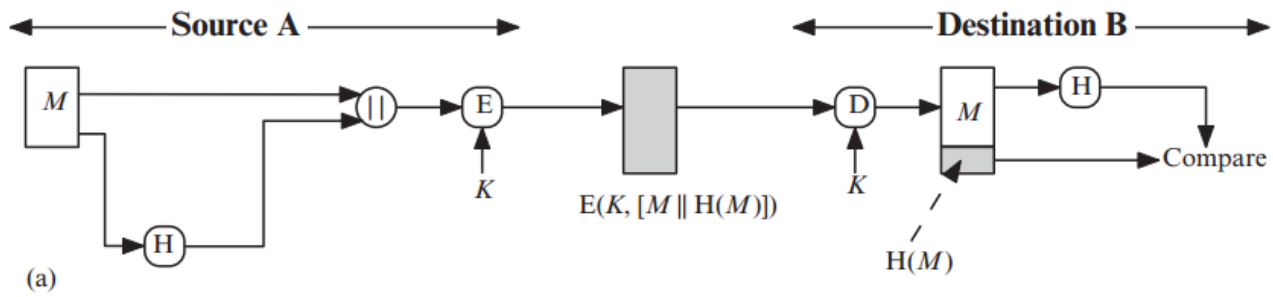
- RSA / DSA / ECDSA
- Signature = Created with **private key**
- Verified using **public key**

💡 One-line to remember:

"Digital signature = private key to sign, public key to verify."

Let me know if you want a mnemonic or diagram shortcut too!

Hash Function



Sure! This image shows two diagrams (a) and (b) for **message authentication using encryption and hashing**. Here's a simple explanation you can memorize for each:

✓ Diagram (a): Encrypt-then-MAC

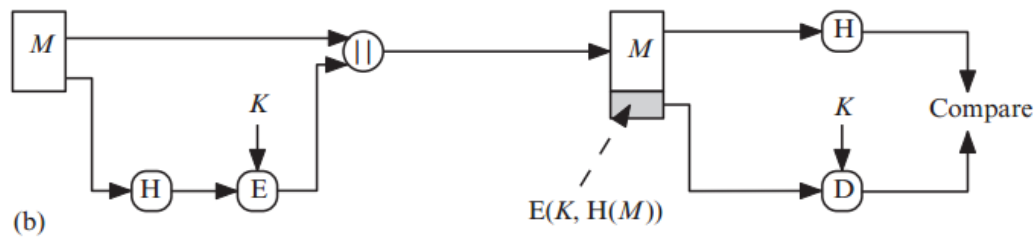
This is when we **encrypt both the message and its hash**.

🔄 Steps at Source A (Sender):

1. M = Message.
2. Hash the message $\rightarrow H(M)$.
3. Combine the message and its hash $\rightarrow M || H(M)$.
4. Encrypt the combined data using key $K \rightarrow E(K, M || H(M))$.
5. Send the encrypted data to Destination B.

🔄 Steps at Destination B (Receiver):

1. Decrypt the received message using the same key $K \rightarrow$ Get M and $H(M)$.
2. Hash M again using same hash function.
3. Compare received $H(M)$ with calculated one.
4. If match = Message is authentic ✓
If not = Message was tampered ✗



✓ Diagram (b): MAC-then-Encrypt

This is when we **hash the message, encrypt the hash, and send it with the plain message.**

🔄 Steps at Source A (Sender):

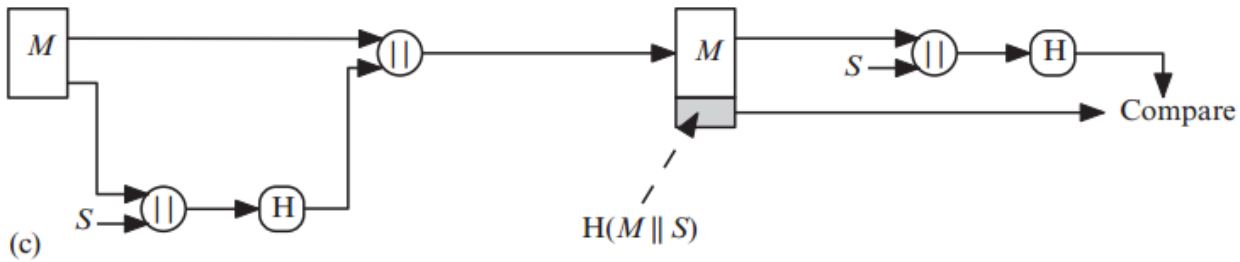
1. **M** = Message.
2. Hash the message $\rightarrow H(M)$.
3. Encrypt the hash using key **K** $\rightarrow E(K, H(M))$.
4. Send both **M** and **$E(K, H(M))$** together.

🔄 Steps at Destination B (Receiver):

1. Hash the received **M** $\rightarrow H(M)$.
2. Decrypt the encrypted hash using key **K** $\rightarrow$ get **$H(M)$** .
3. Compare both hashes.
4. If match = Message is authentic ✓
If not = Message was tampered ✗

🔑 Key Concepts to Remember:

- $H(M)$ = Hash of the message.
- $E(K, \dots)$ = Encryption using key **K**.
- $||$ = Concatenation (joining two things).
- Authentication is done by **comparing hashes**.



✓ **Diagram (c): Secret Value Added to Message Before Hashing**

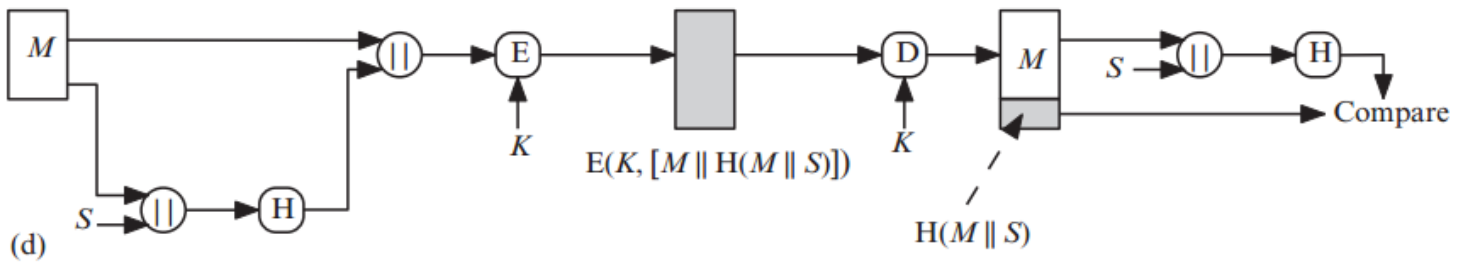
This method uses a **shared secret value S** for authentication.

🔗 **Source A (Sender):**

1. You have the message **M**.
2. You also have a shared secret **S**.
3. Combine message and secret → **M || S**.
4. Hash it → **H(M || S)**.
5. Send both the message **M** and the hash **H(M || S)**.

🔗 **Destination B (Receiver):**

1. Receive **M** and **H(M || S)**.
2. Receiver also knows the secret **S**.
3. Receiver creates the same **M || S**.
4. Hash it again.
5. Compare the two hashes.
 - If match = message is **authentic** ✓
 - If not = **tampered** ✗



✓ **Diagram (d): Encrypt Message and Hash Together with Secret**

This combines **secret-based hashing** with **encryption**.

🔒 **Source A (Sender):**

1. Message is M .
2. Shared secret is S .
3. Make $M || S$, then hash $\rightarrow H(M || S)$.
4. Combine message and hash $\rightarrow M || H(M || S)$.
5. Encrypt all using key $K \rightarrow E(K, [M || H(M || S)])$.
6. Send the encrypted result.

🔒 **Destination B (Receiver):**

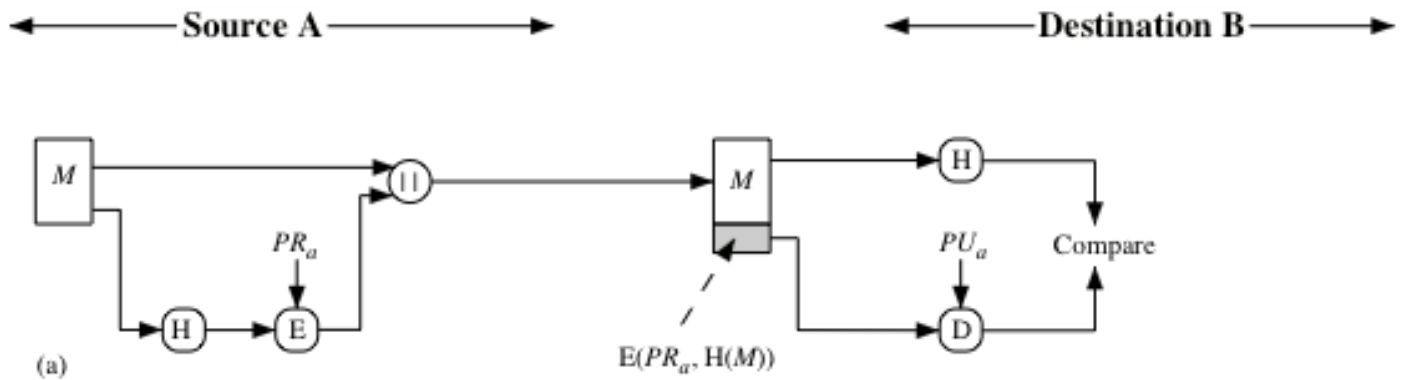
1. Decrypt using same key $K \rightarrow$ Get M and $H(M || S)$.
2. Receiver knows S , so forms $M || S$, then hashes it.
3. Compare the received hash with newly calculated one.
 - If match = message is **authentic** ✓
 - If not = **tampered** ✗

📌 **Quick Memorization Tips:**

- $||$ means joining (concatenation).
- $H(\dots)$ = hash function.
- $E(\dots)$ and $D(\dots)$ = Encrypt and Decrypt with key K .
- S = secret shared between sender and receiver.
- Final step always = **Compare** the hash to check integrity.

Let me know if you want ASCII-style or shortcut drawing ideas for exam revision!

Simplified Digital Signatures



✓ Diagram (e): Secret-then-Hash ($H(S || M)$)

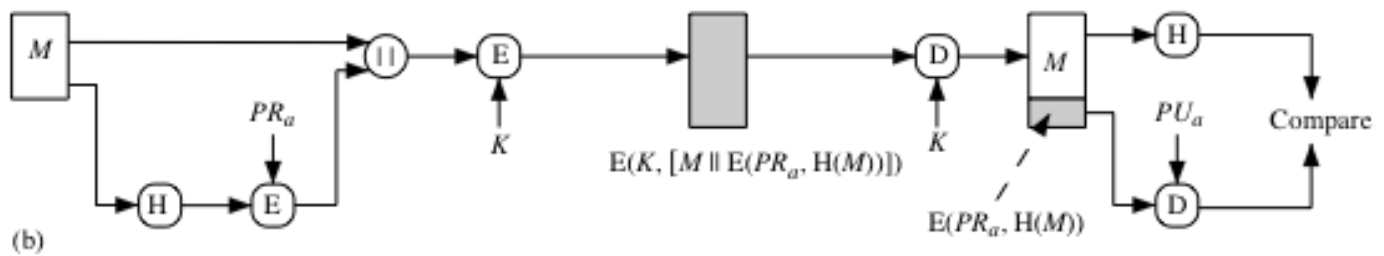
This method uses a **secret S** that is **added before the message**, and then hashed.

🔄 Source A (Sender):

1. You have:
 - Message **M**
 - Shared secret **S**
2. Combine secret and message $\rightarrow S || M$
3. Hash this $\rightarrow H(S || M)$
4. Send **M** and $H(S || M)$ to Destination B.

🔄 Destination B (Receiver):

1. Receiver also knows secret **S**
2. Combine S and received **M** $\rightarrow S || M$
3. Hash it $\rightarrow H(S || M)$
4. Compare with received hash.
 - If match ✓: Message is **authentic**
 - If not ✗: Message is **tampered**



✓ Diagram (f): Encrypt ($M || H(S || M)$)

This adds **encryption on top of the hash** for better security.

🔑 Source A (Sender):

- You have:
 - Message **M**
 - Secret **S**
- Create $S || M \rightarrow \text{Hash} \rightarrow H(S || M)$
- Combine with message $\rightarrow M || H(S || M)$
- Encrypt all using secret key $K \rightarrow E(K, [M || H(S || M)])$
- Send the encrypted result.

🔑 Destination B (Receiver):

- Decrypt using same key $K \rightarrow$ Get **M** and $H(S || M)$
- Receiver also knows secret **S**
- Combine S and M $\rightarrow S || M \rightarrow \text{Hash}$ again
- Compare the two hashes
 - If match ✓: Message is **valid**
 - If not ✗: Message is **changed**

why diagrams (e) and (f) are called "simplified examples of digital signatures" ?

✓ Here's the Simple Explanation:

Diagrams (e) and (f) **simulate the behavior** of digital signatures using **hashing + a secret (S)** and sometimes **encryption (with key K)** — without actually using public/private keys.

They are simplified because:

🔑 What a real digital signature does:

- Uses a **private key** to "sign" a message.
- Anyone with the **public key** can verify the message.
- Ensures:
 - Integrity** (message not changed)
 - Authentication** (who sent it)
 - Non-repudiation** (sender cannot deny)

✓ Why (e) and (f) are simplified digital signatures:

Feature	Diagram (e) / (f)	Real Digital Signature
Uses a hash of the message	✓ Yes, like real signatures	✓ Yes
Uses a secret or key	✓ Uses shared secret or symmetric key	✗ Uses private/public key pair
Ensures integrity	✓ Yes	✓ Yes
Ensures authentication	✓ Yes (with shared secret)	✓ Yes (with private key)
Ensures non-repudiation	✗ No (sender can deny if secret is shared)	✓ Yes (only sender knows private key)

🧠 Conclusion (Memorize this):

"Diagrams (e) and (f) are called **simplified digital signatures** because they use hashing and secret keys to achieve message integrity and authentication — similar to digital signatures — but without using public-key cryptography."